08 — Experiment Tracking and MLOps Lite Plan

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	MLO-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	Technical Design, Modeling Roadmap, Roadmap

---

1. Philosophy

This project uses a **lightweight, file-based MLOps approach** appropriate for a small team working on Kaggle. No complex infrastructure (MLflow servers, Kubernetes, cloud pipelines) is required. The system should be:

- **Simple**: File-based logging that any team member can read and update.
- **Reproducible**: Every result can be re-derived from logged metadata.
- **Portable**: Works identically on Kaggle, local machines, and Google Colab.
- **Evolvable**: Can be upgraded to W&B or MLflow later without losing historical data.

---

2. Folder Conventions

2.1 Top-Level Structure

```
project-root/
■■■ data/
■   ■■■ raw/                          # Downloaded source files (never modified)
■   ■   ■■■ encode/                   # ENCODE BED/narrowPeak files
■   ■   ■■■ reference/                # hg38 FASTA files
■   ■   ■■■ motifs/                   # JASPAR PFMs
■   ■   ■■■ annotations/             # GENCODE GTF
■   ■■■ processed/                    # Processed/cleaned data
■   ■   ■■■ sequences/               # Extracted DNA sequences
■   ■   ■■■ labels/                  # Label files with splits
■   ■■■ features/                     # Cached feature matrices
■   ■   ■■■ kmer_k4_v0.1.0.npz
■   ■   ■■■ kmer_k5_v0.1.0.npz
■   ■   ■■■ onehot_v0.1.0.npz
■   ■   ■■■ gc_dinuc_v0.1.0.npz
■   ■■■ splits/                       # Split definitions
■       ■■■ chrom_holdout_v0.1.0.json
■■■ src/                              # Source code modules
■■■ notebooks/                        # Exploration and Kaggle notebooks
■■■ experiments/                      # ALL experiment outputs go here
■   ■■■ logs/                         # Experiment metadata and metrics
```

```
■   ■   ■■■ exp_001_logistic_kmer4/
■   ■   ■   ■■■ config.yaml
■   ■   ■   ■■■ metrics.json
■   ■   ■   ■■■ notes.md
■   ■   ■■■ exp_002_rf_kmer456/
■   ■       ■■■ config.yaml
■   ■       ■■■ metrics.json
■   ■       ■■■ notes.md
■   ■■■ models/                             # Saved model artifacts
■   ■   ■■■ exp_001_logistic_kmer4/
■   ■   ■   ■■■ model.pkl
■   ■   ■■■ exp_002_rf_kmer456/
■   ■       ■■■ model.pkl
■   ■■■ figures/                            # Generated plots
■   ■   ■■■ exp_001_logistic_kmer4/
■   ■   ■   ■■■ confusion_matrix.png
■   ■   ■   ■■■ roc_curve.png
■   ■   ■■■ exp_002_rf_kmer456/
■   ■       ■■■ confusion_matrix.png
■   ■       ■■■ roc_curve.png
■   ■       ■■■ feature_importance.png
■   ■       ■■■ shap_summary.png
■   ■■■ reports/                            # Generated summary reports
■       ■■■ model_comparison_v0.1.0.md
■■■ configs/                                # Configuration files
■■■ tests/                                  # Unit tests
■■■ docs/                                   # Documentation (this pack)
```

2.2 Rules

1. **`data/raw/` is read-only** after initial download. Never modify raw files.
2. **`data/processed/` and `data/features/`** are derived from raw data. They can be regenerated by re-running the pipeline.
3. **`experiments/`** is the single source of truth for all experiment outputs.
4. **Every experiment gets its own subdirectory** under `experiments/logs/`, `experiments/models/`, and `experiments/figures/`.
5. **No experiment outputs in `notebooks/`**. Notebooks save outputs to `experiments/`.

---

3. Experiment Naming Convention

3.1 Format

```
exp_{NNN}_{model}_{features}[_{variant}]
```

Component	Description	Examples
`NNN`	Three-digit sequential number	`001`, `002`, `015`
`model`	Model family short name	`logistic`, `rf`, `xgb`, `cnn`, `pretrained`
`features`	Feature set short name	`kmer4`, `kmer456`, `kmer_gc`, `onehot`, `dnabert2`
`variant`	Optional: distinguishes sub-experiments	`gcmatch`, `flanking`, `k562`, `gm12878`

3.2 Examples

```
exp_001_logistic_kmer4
exp_002_rf_kmer456
exp_003_xgb_kmer_gc_motif
exp_004_rf_kmer456_flanking_neg
exp_005_cnn_onehot
exp_006_cnn_onehot_k562
exp_007_cnn_onehot_gm12878
exp_008_pretrained_dnabert2_embed
exp_009_xgb_kmer_dnabert2_hybrid
```

3.3 Naming Registry

Maintain a running experiment registry in `experiments/REGISTRY.md`:

```
# Experiment Registry

| ID | Name | Phase | Status | Key result |
|----|------|-------|--------|------------|
| 001 | exp_001_logistic_kmer4 | 1 | Complete | AUROC = 0.72 |
| 002 | exp_002_rf_kmer456 | 1 | Complete | AUROC = 0.81 |
| 003 | exp_003_xgb_kmer_gc_motif | 1 | Running | — |
```

---

4. Notebook Hygiene

4.1 Notebook Header Template

Every notebook must begin with a metadata cell:

```
# ============================================
# Experiment: exp_002_rf_kmer456
# Date: 2026-06-15
# Author: [team member name]
# Purpose: Train random forest on k-mer (k=4,5,6) features
# Data version: cCRE_v3_2026-06-07 / hg38_GRCh38.p14
# Feature version: kmer_k456_v0.1.0
# Split version: chrom_holdout_v0.1.0
# Random seed: 42
# Environment: Kaggle GPU T4 / Python 3.10
# ============================================
```

4.2 Notebook Structure Rules

Rule	Rationale
1. Header cell with metadata	Reproducibility; experiment identification
2. Imports cell	All imports at the top; explicit versions logged
3. Configuration cell	All hyperparameters defined in one place (dict or YAML)
4. Data loading cell	Load from versioned paths; print shapes and checksums
5. Modeling cells	Train, evaluate, interpret — in clearly separated sections
6. Logging cell	Save config, metrics, and artifacts to `experiments/`
7. Summary cell	Print key results; compare to previous experiments

8. Clear outputs before commit	Keep repository clean; outputs are in `experiments/`

4.3 Kaggle-Specific Notebook Rules

Rule	Detail
Self-contained	Kaggle notebooks must include or import all required code
Dataset references	Use Kaggle dataset paths (`/kaggle/input/...`)
Output saving	Save to `/kaggle/working/` (auto-persisted as notebook output)
Install dependencies	`!pip install` at top if needed; comment with version
GPU check	Include a cell that checks GPU availability and prints device info

---

5. Versioning Strategy

5.1 Data Versioning

What	Version format	Example
ENCODE download	`{source}_{release}_{download_date}`	`cCRE_v3_2026-06-07`
Reference genome	`{assembly}_{patch}`	`hg38_GRCh38.p14`
Processed dataset	Semantic version	`processed_v0.1.0`
Feature cache	`{feature_type}_{config_hash}`	`kmer_k456_v0.1.0`
Split definition	Semantic version	`chrom_holdout_v0.1.0`

5.2 Code Versioning

- **Git** for all source code and notebooks (with outputs cleared).
- **Branching strategy**: `main` (stable) + `dev` (active development) + feature branches.
- **Commit messages**: Conventional commits format (`feat:`, `fix:`, `data:`, `exp:`, `docs:`).
- **Tags**: Tag releases that correspond to major experiment milestones (e.g., `v0.1.0-baseline`).

5.3 Model Versioning

Model artifact	Location	Naming
Trained model	`experiments/models/{exp_name}/model.pkl`	One model per experiment directory
Model config	`experiments/logs/{exp_name}/config.yaml`	Hyperparameters that produced the model
Model metrics	`experiments/logs/{exp_name}/metrics.json`	Evaluation metrics

---

6. Metrics Logging

6.1 Metrics File Format

Each experiment saves a `metrics.json`:

```json
{
  "experiment_name": "exp_002_rf_kmer456",
  "timestamp": "2026-06-15T14:30:00Z",
  "model_family": "random_forest",
  "features": "kmer_k456",
  "dataset_version": "cCRE_v3_2026-06-07",
  "split_version": "chrom_holdout_v0.1.0",
  "random_seed": 42,
  "hyperparameters": {
    "n_estimators": 500,
    "max_depth": 20,
    "min_samples_split": 5,
    "class_weight": "balanced"
  },
  "metrics": {
    "test": {
      "auroc": 0.812,
      "auprc": 0.795,
      "accuracy": 0.763,
      "f1_macro": 0.758,
      "precision": 0.771,
      "recall": 0.745
    },
    "validation": {
      "auroc": 0.823,
      "auprc": 0.808,
      "accuracy": 0.772,
      "f1_macro": 0.768,
      "precision": 0.780,
      "recall": 0.756
    },
    "cv_5fold": {
      "auroc_mean": 0.819,
      "auroc_std": 0.012
    }
  },
  "training_time_seconds": 342,
  "peak_memory_mb": 4200,
  "environment": {
    "platform": "kaggle",
    "gpu": "T4",
    "python_version": "3.10.12",
    "sklearn_version": "1.3.2"
  }
}
```

6.2 Configuration File Format

Each experiment saves a `config.yaml`:

```yaml
experiment_name: "exp_002_rf_kmer456"
model:
  family: "random_forest"
  n_estimators: 500
  max_depth: 20
  min_samples_split: 5
  class_weight: "balanced"
data:
  dataset_version: "cCRE_v3_2026-06-07"
  cell_type: "K562"
  element_types: ["dELS", "PLS"]
  negative_strategy: "gc_matched_exclude_known"
```

```yaml
  positive_negative_ratio: 1.0
features:
  type: "kmer"
  k_values: [4, 5, 6]
  include_gc: true
  include_dinucleotide: true
split:
  version: "chrom_holdout_v0.1.0"
  train_chroms: ["chr1", "chr2", "chr3", "chr4", "chr5", "chr6", "chr7",
                 "chr8", "chr9", "chr10", "chr11", "chr12", "chr13", "chr14", "chrX"]
  val_chroms: ["chr15", "chr16", "chr17", "chr18"]
  test_chroms: ["chr19", "chr20", "chr21", "chr22"]
random_seed: 42
```

---

7. Model Artifact Storage

7.1 What to Save

Artifact	Format	Always save?
Trained model	`.pkl` (scikit-learn), `.pt` (PyTorch), `.json` (XGBoost)	Yes
Hyperparameters	`config.yaml`	Yes
Evaluation metrics	`metrics.json`	Yes
Feature importance	`feature_importance.csv`	Yes
Confusion matrix plot	`confusion_matrix.png`	Yes
ROC curve plot	`roc_curve.png`	Yes
SHAP summary plot	`shap_summary.png`	If SHAP was computed
Saliency maps	`saliency_sample_{N}.png`	If saliency was computed
Training logs (loss curves)	`training_log.csv`	For neural models
Experiment notes	`notes.md`	Recommended

7.2 What NOT to Save to Git

Artifact	Reason	Alternative
Raw data files	Too large	Document download procedure; cache on Kaggle
Feature matrices (.npz)	Derived; can be recomputed	Document computation procedure
Trained model binaries (large)	Can be 100 MB+	Store in Kaggle output or cloud storage
Reference genome FASTA	3+ GB	Document download source

7.3 .gitignore Template

```
# Data
data/raw/
data/reference/
data/processed/
data/features/

# Large model artifacts
experiments/models/**/*.pkl
experiments/models/**/*.pt
experiments/models/**/*.json

# Notebook outputs
notebooks/**/.ipynb_checkpoints/

# Python
__pycache__/
*.pyc
.env

# OS
.DS_Store
Thumbs.db
```

8. Reproducibility Checklist

Before declaring any experiment "complete," verify:

#	Check	How
1	Random seed is set and logged	Check `config.yaml` → `random_seed: 42`
2	Data version is logged	Check `config.yaml` → `dataset_version`
3	Split version is logged	Check `config.yaml` → `split.version`
4	All hyperparameters are logged	Compare `config.yaml` to code
5	Software versions are logged	Check `metrics.json` → `environment`
6	Metrics are saved	Check `metrics.json` exists and is non-empty
7	Feature computation is deterministic	Re-run feature extraction; verify checksums match
8	Model can be loaded and re-evaluated	Load saved model; predict on test set; verify metrics match
9	Notebook runs top-to-bottom	Clear outputs and re-run; verify success
10	No hardcoded absolute paths (except Kaggle `/kaggle/input/`)	Grep for local paths

Reproducibility Verification Command

```
# Quick verification script
import json
import hashlib

def verify_experiment(exp_dir):
    """Check that an experiment is reproducible."""
    checks = []

    # Check config exists
    config_path = f"{exp_dir}/config.yaml"
    checks.append(("config exists", os.path.exists(config_path)))

    # Check metrics exist
    metrics_path = f"{exp_dir}/metrics.json"
    checks.append(("metrics exist", os.path.exists(metrics_path)))

    # Check metrics are valid JSON
    try:
        with open(metrics_path) as f:
            metrics = json.load(f)
        checks.append(("metrics valid JSON", True))
        checks.append(("has AUROC", "auroc" in metrics.get("metrics", {}).get("test", {})))
        checks.append(("has random_seed", "random_seed" in metrics))
    except:
        checks.append(("metrics valid JSON", False))

    for name, passed in checks:
        status = "✓" if passed else "✗"
        print(f"  {status} {name}")

    return all(passed for _, passed in checks)
```

---

9. Upgrading to External Tracking (Optional, Later)

9.1 When to Upgrade

Trigger	Recommended tool
> 20 experiments and comparisons become unwieldy	Weights & Biases (free tier)
Multiple team members running experiments simultaneously	W&B or MLflow
Need interactive experiment comparison dashboards	W&B
Need model registry with staging/production labels	MLflow

9.2 Migration Path

The file-based logging format is designed to be easily importable:

```
# Example: migrate to W&B
import wandb
import json
import yaml

for exp_dir in sorted(glob("experiments/logs/exp_*")):
    config = yaml.safe_load(open(f"{exp_dir}/config.yaml"))
    metrics = json.load(open(f"{exp_dir}/metrics.json"))

    run = wandb.init(project="regulatory-switch", name=config["experiment_name"], config=config)
    run.log(metrics["metrics"]["test"])
```

```
    run.finish()
```

End of Document 08 — Experiment Tracking and MLOps Lite Plan
Next: 09 — Risk Register